Kubernetes — 3–4 YOE Scenario-Based Q&A

1. Kubernetes Pod is in CrashLoopBackOff. What will you check?
Ans: First, I check the Pod status and container logs to understand why the application is repeatedly stopping. I also check the previous container logs because the current container may have restarted already.
Then I verify the container command, environment variables, ConfigMaps, Secrets, and application configuration. I also check whether the container is failing because of an application error, health check failure, or resource issue.
Simple ga gurthupettuko: Pod CrashLoopBackOff lo unte → first Pod status and logs chudu → previous logs check cheyyi → command and configuration verify cheyyi → ConfigMap and Secret chudu → health check or resource issue unda identify cheyyi → root cause fix chesi Pod ni verify cheyyi.
Eng: If the Pod is in CrashLoopBackOff → first check Pod status and logs → check previous logs → verify command and configuration → check ConfigMap and Secret → identify health check or resource issues → fix the root cause and verify the Pod.
Example:
kubectl get pods → Pod CrashLoopBackOff lo unda chudu.
kubectl logs <pod-name> → current container error em vastundo chudu.
kubectl logs <pod-name> --previous → previous container enduku restart ayyindo chudu.
kubectl describe pod <pod-name> → Events lo error, probe failure, or configuration issue unda check cheyyi.
Issue fix chesi → kubectl get pods → Pod Running and Ready ga ayyinda verify cheyyi.

---

2. Kubernetes Pod is in ImagePullBackOff. What will you check?
Ans: First, I check the Pod events and identify why Kubernetes is unable to pull the image. Then I verify the image name, repository, and tag specified in the Deployment or Pod configuration.
If the image is from a private registry, I check whether the required image pull Secret is configured correctly. I also verify registry access and whether the requested image actually exists.
Simple ga gurthupettuko: Pod ImagePullBackOff lo unte → first events chudu → image name, repository, tag verify cheyyi → private registry ayithe imagePullSecret check cheyyi → registry access verify cheyyi → image actually unda chudu → issue fix chesi Pod ni recreate cheyyi.
Eng: If the Pod is in ImagePullBackOff → first check events → verify image name, repository, and tag → check imagePullSecret for a private registry → verify registry access → confirm the image exists → fix the issue and recreate the Pod.
Example:
kubectl describe pod <pod-name> → Events lo image pull error em vastundo chudu.
kubectl get pod <pod-name> -o yaml → image name and tag correct ga unnaya check cheyyi.
docker pull <image-name>:<tag> → image registry nunchi pull avutunda verify cheyyi.
kubectl get secret → required imagePullSecret unda chudu.
Issue fix chesi → kubectl delete pod <pod-name> → new Pod image pull chesi Running ayyinda verify cheyyi.

---

3. Kubernetes Pod is stuck in Pending state. What will you check?
Ans: First, I check the Pod events to understand why the scheduler is not placing the Pod on a node. Then I verify node availability, resource requests, node selectors, taints, and tolerations.
I also check whether the Pod has any affinity or anti-affinity rules that are preventing scheduling. After identifying the scheduling issue, I make the required correction and verify that the Pod gets scheduled.
Simple ga gurthupettuko: Pod Pending lo unte → first Pod events chudu → scheduler enduku place cheyyaledo identify cheyyi → nodes available unnaya check cheyyi → CPU and memory requests verify cheyyi → node selector, taints and tolerations chudu → scheduling issue fix chesi Pod schedule ayyinda verify cheyyi.
Eng: If the Pod is Pending → first check Pod events → identify why the scheduler cannot place it → verify node availability → check CPU and memory requests → verify node selectors, taints, and tolerations → fix the scheduling issue and verify the Pod.
Example:
kubectl get pods → Pod Pending lo unda chudu.
kubectl describe pod <pod-name> → Events lo FailedScheduling reason chudu.
kubectl get nodes → available nodes unnaya check cheyyi.
kubectl describe node <node-name> → CPU, memory, taints and node conditions check cheyyi.
Issue fix chesi → kubectl get pods → Pod node ki schedule ayyinda verify cheyyi.

---

4. Kubernetes Pod is OOMKilled. What will you check?
Ans: First, I check the Pod status and container details to confirm that the container was terminated because it exceeded its memory limit. Then I check the memory requests and limits configured for the container.
I also review the application logs and memory usage to understand whether the application is consuming more memory than expected or has a possible memory leak. Based on the root cause, I either fix the application issue or adjust the memory configuration appropriately.
Simple ga gurthupettuko: Pod OOMKilled ayithe → first memory valla terminate ayyinda confirm cheyyi → memory usage chudu → requests and limits verify cheyyi → application logs check cheyyi → memory leak or high usage unda identify cheyyi → root cause fix chesi memory configuration verify cheyyi.
Eng: If the Pod is OOMKilled → first confirm the memory termination → check memory usage → verify requests and limits → check application logs → identify high usage or memory leak → fix the root cause and verify the memory configuration.
Example:
kubectl describe pod <pod-name> → OOMKilled reason unda chudu.
kubectl top pod <pod-name> → Pod memory usage entha undo check cheyyi.
kubectl get pod <pod-name> -o yaml → memory requests and limits verify cheyyi.
kubectl logs <pod-name> --previous → restart mundu application error emaina unda chudu.
Memory issue fix chesi → kubectl top pod <pod-name> → memory usage normal ga unda verify cheyyi.

5. Kubernetes Service is not working. What will you check?
Ans: First, I check whether the Service exists and verify its configuration, especially the Service port, target port, and selector. Then I check whether the selector is matching the required Pod labels.
I also verify whether the backend Pods are running and whether the Service has valid endpoints. Finally, I test connectivity to the Service and check the application logs if required.
Simple ga gurthupettuko: Service work avvakapothe → first Service configuration chudu → port and targetPort verify cheyyi → selector and Pod labels match avutunnaya check cheyyi → backend Pods running unnaya chudu → endpoints create ayyaya verify cheyyi → connectivity test cheyyi.
Eng: If the Service is not working → first check Service configuration → verify port and targetPort → check selector and Pod labels → verify backend Pods are running → check Service endpoints → test connectivity.
Example:
kubectl get svc <service-name> → Service exist ayyi ports correct ga unnaya chudu.
kubectl describe svc <service-name> → selector and targetPort verify cheyyi.
kubectl get pods --show-labels → Service selector ki Pod labels match avutunnaya check cheyyi.
kubectl get endpoints <service-name> → backend Pod endpoints unnaya chudu.
kubectl exec -it <pod-name> -- curl <service-name>:<port> → Service ki connectivity work avutunda test cheyyi.

---

6. Kubernetes Ingress is not working. What will you check?
Ans: First, I check whether the Ingress resource is created correctly and whether the Ingress Controller is running. Then I verify the hostname, path rules, Service name, and Service port configured in the Ingress.
I also check the Ingress Controller logs and DNS configuration to confirm that the hostname is resolving to the correct endpoint. If TLS is configured, I verify the certificate and Secret as well.
Simple ga gurthupettuko: Ingress work avvakapothe → first Ingress configuration chudu → Ingress Controller running unda check cheyyi → hostname and path rules verify cheyyi → Service name and port check cheyyi → DNS and Controller logs chudu → TLS unte certificate and Secret verify cheyyi.
Eng: If the Ingress is not working → first check Ingress configuration → verify the Ingress Controller → check hostname and path rules → verify Service name and port → check DNS and Controller logs → verify the certificate and Secret if TLS is enabled.
Example:
kubectl get ingress → Ingress created unda and address vachinda chudu.
kubectl describe ingress <ingress-name> → hostname, path, Service and port correct ga unnaya check cheyyi.
kubectl get pods -n ingress-nginx → Ingress Controller Pods running unnaya chudu.
kubectl logs -n ingress-nginx <controller-pod> → Controller side error emaina unda check cheyyi.
nslookup <hostname> → hostname correct endpoint ki resolve avutunda verify cheyyi.

---

7. Kubernetes Pod cannot resolve a service name. What will you check?
Ans: First, I check whether the DNS-related components such as CoreDNS are running correctly. Then I verify the Service name, namespace, and DNS name being used by the application.
I also check whether the Pod can reach the cluster DNS service and whether any NetworkPolicy is blocking DNS traffic. Finally, I test DNS resolution from inside the Pod.
Simple ga gurthupettuko: Service name resolve avvakapothe → first CoreDNS running unda chudu → Service name and namespace verify cheyyi → correct DNS name use chesthunnara check cheyyi → Pod nunchi DNS service reach avutunda chudu → NetworkPolicy block chesthunda verify cheyyi → Pod lopala DNS test cheyyi.
Eng: If the service name cannot be resolved → first check CoreDNS → verify Service name and namespace → confirm the correct DNS name → check Pod access to cluster DNS → verify NetworkPolicy restrictions → test DNS from inside the Pod.
Example:
kubectl get pods -n kube-system → CoreDNS Pods Running lo unnaya chudu.
kubectl get svc <service-name> → Service name and namespace correct aa check cheyyi.
kubectl exec -it <pod-name> -- nslookup <service-name> → Pod lopala Service name resolve avutunda test cheyyi.
kubectl get networkpolicy → DNS traffic ni NetworkPolicy block chesthunda check cheyyi.
Issue fix chesi → nslookup <service-name> → Service name resolve ayyinda verify cheyyi.

---

8. Kubernetes readiness probe is failing. What will you check?
Ans: First, I check the Pod events and container logs to understand why the readiness probe is failing. Then I verify the probe path, port, protocol, initial delay, timeout, and failure threshold.
I also confirm that the application is actually listening on the configured port and that the readiness endpoint is returning the expected response. Since readiness controls whether the Pod receives traffic, I verify the Service endpoints as well.
Simple ga gurthupettuko: Readiness probe fail ayithe → first events and logs chudu → probe path and port verify cheyyi → protocol, delay and timeout check cheyyi → application expected port lo listen chesthunda chudu → health endpoint response verify cheyyi → Service traffic receive chesthunda check cheyyi.
Eng: If the readiness probe fails → first check events and logs → verify probe path and port → check protocol, delay, and timeout → confirm the application is listening correctly → verify the health endpoint response → check Service traffic.
Example:
kubectl describe pod <pod-name> → Events lo readiness probe failure reason chudu.
kubectl get pod <pod-name> -o yaml → readiness path, port, delay and timeout verify cheyyi.
kubectl exec -it <pod-name> -- curl localhost:<port>/<health-path> → health endpoint correct response isthunda check cheyyi.
kubectl get endpoints <service-name> → Ready Pod Service endpoint lo unda chudu.
Probe configuration or application issue fix chesi → kubectl get pods → Pod Ready ayyinda verify cheyyi.

9. Kubernetes liveness probe is failing. What will you check?
Ans: First, I check the Pod events and logs to understand why the liveness probe is failing. Then I verify the liveness probe path, port, protocol, initial delay, timeout, and failure threshold.
I also check whether the application is actually healthy or whether the probe configuration is incorrect. If the application is healthy but the probe is too aggressive, I correct the probe settings and verify that the Pod stops restarting.
Simple ga gurthupettuko: Liveness probe fail ayithe → first events and logs chudu → probe path, port and protocol verify cheyyi → delay and timeout check cheyyi → application actually healthy ga unda chudu → probe configuration wrong aa identify cheyyi → correct chesi Pod restart avvadam aaginda verify cheyyi.
Eng: If the liveness probe fails → first check events and logs → verify probe path, port, and protocol → check delay and timeout → confirm application health → identify incorrect probe settings → fix them and verify the Pod stops restarting.
Example:
kubectl describe pod <pod-name> → liveness probe failure reason chudu.
kubectl get pod <pod-name> -o yaml → liveness path, port, delay and timeout verify cheyyi.
kubectl exec -it <pod-name> -- curl localhost:<port>/<health-path> → application health endpoint work avutunda check cheyyi.
kubectl get pods → restart count increase avutunda chudu.
Probe issue fix chesi → kubectl get pods → Pod unnecessary ga restart avvadam aaginda verify cheyyi.

---

10. Kubernetes Node is NotReady. What will you check?
Ans: First, I check the node status and describe the node to identify the reason for the NotReady condition. Then I check node conditions, kubelet status, container runtime, CPU, memory, and disk usage.
I also check whether there are network connectivity issues between the node and the control plane. After identifying the problem, I fix the node issue and verify that the node becomes Ready again.
Simple ga gurthupettuko: Node NotReady ayithe → first node status and conditions chudu → kubelet check cheyyi → container runtime verify cheyyi → CPU, memory and disk usage chudu → control plane connectivity check cheyyi → node issue fix chesi Ready status verify cheyyi.
Eng: If the node is NotReady → first check node status and conditions → verify kubelet → check the container runtime → check CPU, memory, and disk usage → verify control plane connectivity → fix the node issue and confirm Ready status.
Example:
kubectl get nodes → ye Node NotReady lo undo identify cheyyi.
kubectl describe node <node-name> → node conditions and events lo reason chudu.
systemctl status kubelet → kubelet service running unda check cheyyi.
systemctl status containerd → container runtime running unda verify cheyyi.
df -h and free -m → disk and memory usage check cheyyi.
Issue fix chesi → kubectl get nodes → Node Ready ayyinda verify cheyyi.

---

11. Kubernetes Deployment rollout is failing. What will you check?
Ans: First, I check the Deployment status and rollout history to identify which ReplicaSet or Pod is failing. Then I check the Pod events and container logs for image, configuration, scheduling, or application errors.
I also verify the Deployment strategy, resource configuration, probes, and image version. After fixing the issue, I monitor the rollout and confirm that the required number of replicas become available.
Simple ga gurthupettuko: Deployment rollout fail ayithe → first rollout status and history chudu → failed ReplicaSet or Pod identify cheyyi → Pod events and logs check cheyyi → image, configuration and probes verify cheyyi → resources and strategy chudu → issue fix chesi rollout successful ga complete ayyinda verify cheyyi.
Eng: If the Deployment rollout fails → first check rollout status and history → identify the failed ReplicaSet or Pod → check Pod events and logs → verify image, configuration, and probes → check resources and strategy → fix the issue and verify the rollout.
Example:
kubectl rollout status deployment/<deployment-name> → rollout ekkada stuck ayyindo chudu.
kubectl rollout history deployment/<deployment-name> → previous revisions check cheyyi.
kubectl get pods → new Pods lo error unda chudu.
kubectl describe pod <pod-name> → Events lo failure reason check cheyyi.
Issue fix chesi → kubectl rollout status deployment/<deployment-name> → rollout complete ayyinda verify cheyyi.

---

12. How will you rollback a failed Kubernetes deployment?
Ans: First, I confirm that the latest Deployment version is causing the issue by checking the rollout status, Pod health, and application behavior. Then I review the Deployment rollout history to identify the last known good revision.
I rollback to the previous stable revision and monitor the new Pods. After the rollback, I verify the application, Service, and Pod health to make sure the production issue is resolved.
Simple ga gurthupettuko: Deployment fail ayithe → first latest version problem aa confirm cheyyi → rollout history lo last stable revision chudu → previous revision ki rollback cheyyi → new Pods health check cheyyi → application and Service verify cheyyi → issue resolve ayyinda confirm cheyyi.
Eng: If the Deployment fails → first confirm the latest version is causing the issue → find the last stable revision → rollback to the previous revision → check new Pod health → verify the application and Service → confirm the issue is resolved.
Example:
kubectl rollout status deployment/<deployment-name> → current rollout problem unda chudu.
kubectl rollout history deployment/<deployment-name> → last successful revision identify cheyyi.
kubectl rollout undo deployment/<deployment-name> → previous stable version ki rollback cheyyi.
kubectl rollout status deployment/<deployment-name> → rollback successful ga complete ayyinda verify cheyyi.
kubectl get pods → new Pods healthy ga unnaya check cheyyi.

13. Kubernetes Pods are not scaling as expected. What will you check?
Ans: First, I check whether the Horizontal Pod Autoscaler is configured correctly and whether it is receiving the required metrics. Then I verify the minimum and maximum replica settings and the target CPU or memory utilization.
I also check whether resource requests are configured because HPA may depend on them for utilization calculations. Finally, I check HPA events and metrics to understand why scaling is not happening.
Simple ga gurthupettuko: Pods expected ga scale avvakapothe → first HPA configuration chudu → metrics available unnaya check cheyyi → min and max replicas verify cheyyi → CPU or memory target check cheyyi → resource requests configured unnaya chudu → HPA events and metrics verify cheyyi.
Eng: If Pods are not scaling as expected → first check HPA configuration → verify metrics availability → check minimum and maximum replicas → verify CPU or memory targets → check resource requests → review HPA events and metrics.
Example:
kubectl get hpa → HPA target and current metrics chudu.
kubectl describe hpa <hpa-name> → scaling events and errors check cheyyi.
kubectl top pods → actual CPU and memory usage chudu.
kubectl get deployment <deployment-name> -o yaml → resource requests configured unnaya verify cheyyi.
Issue fix chesi → kubectl get hpa → desired replicas increase or decrease avutunnaya verify cheyyi.

---

14. Kubernetes ConfigMap or Secret changes are not reflected in the Pod. What will you check?
Ans: First, I check whether the ConfigMap or Secret was actually updated and whether the Pod is referencing the correct resource. Then I verify whether the values are being consumed as environment variables or mounted as files.
If the values are provided through environment variables, I verify whether the Pod needs to be restarted for the new values to take effect. I also check that the correct namespace and resource name are being used.
Simple ga gurthupettuko: ConfigMap or Secret change reflect avvakapothe → first resource update ayyinda chudu → correct ConfigMap or Secret reference unda check cheyyi → environment variable aa file mount aa verify cheyyi → environment variable ayithe Pod restart requirement chudu → namespace and resource name verify cheyyi.
Eng: If ConfigMap or Secret changes are not reflected → first verify the resource update → check the correct reference → verify environment variables or file mounts → check whether a Pod restart is required → verify namespace and resource name.
Example:
kubectl get configmap <configmap-name> -o yaml → latest value update ayyinda chudu.
kubectl get secret <secret-name> -o yaml → Secret correct ga update ayyinda verify cheyyi.
kubectl describe pod <pod-name> → correct ConfigMap or Secret reference unda check cheyyi.
kubectl exec -it <pod-name> -- env → environment variable value updated ayyinda chudu.
Environment variable update avvakapothe → kubectl rollout restart deployment/<deployment-name> → new Pod lo latest value reflect ayyinda verify cheyyi.

---

15. Kubernetes Persistent Volume is not mounting. What will you check?
Ans: First, I check the PersistentVolumeClaim status and identify whether it is Bound or Pending. Then I verify the StorageClass, PersistentVolume, access mode, storage capacity, and volume configuration.
I also check Pod events and node-related issues to understand why the volume cannot be mounted. After correcting the storage configuration, I verify that the PVC becomes Bound and the Pod can mount the volume successfully.
Simple ga gurthupettuko: PV mount avvakapothe → first PVC status chudu → Bound or Pending identify cheyyi → StorageClass and PV check cheyyi → access mode and storage capacity verify cheyyi → Pod events chudu → volume configuration correct cheyyi → PVC Bound ayyi Pod volume mount ayyinda verify cheyyi.
Eng: If the Persistent Volume is not mounting → first check PVC status → identify Bound or Pending → verify StorageClass and PV → check access mode and storage capacity → review Pod events → correct the volume configuration → confirm the PVC is Bound and the volume is mounted.
Example:
kubectl get pvc → PVC Bound or Pending lo undo chudu.
kubectl describe pvc <pvc-name> → Events lo volume binding issue emaina unda check cheyyi.
kubectl get pv → required PV available unda verify cheyyi.
kubectl get storageclass → correct StorageClass available unda chudu.
kubectl describe pod <pod-name> → volume mount error emaina unda check cheyyi.
Issue fix chesi → kubectl get pvc → PVC Bound ayyinda and Pod volume mount ayyinda verify cheyyi.

---

16. Kubernetes Pod is running, but traffic is not reaching the application. What will you check?
Ans: First, I check whether the application is listening on the expected container port. Then I verify the Service selector and confirm that the Service has the correct Pod endpoints.
I also check readiness probe status because a Pod that is not Ready may not receive Service traffic. Finally, I verify the Service port, target port, and network policies if traffic is still not reaching the application.
Simple ga gurthupettuko: Pod running unna traffic raakapothe → first application container port check cheyyi → Service selector and Pod labels verify cheyyi → endpoints create ayyaya chudu → readiness status check cheyyi → Service port and targetPort verify cheyyi → NetworkPolicy issue unda check cheyyi.
Eng: If the Pod is running but traffic is not reaching it → first check the container port → verify Service selector and Pod labels → check endpoints → verify readiness status → check Service port and targetPort → verify NetworkPolicy restrictions.
Example:
kubectl get pods → Pod Running and Ready lo unda chudu.
kubectl get svc <service-name> → Service port and targetPort verify cheyyi.
kubectl get endpoints <service-name> → Pod endpoint Service ki attach ayyinda check cheyyi.
kubectl exec -it <pod-name> -- curl localhost:<port> → application container lopala response vastunda chudu.
kubectl get networkpolicy → traffic ni NetworkPolicy block chesthunda check cheyyi.
Issue fix chesi → Service ki request send chesi → application response vastunda verify cheyyi.

17. Kubernetes application is restarting after deployment. What will you check?
Ans: First, I check the Pod status, restart count, and container logs to identify why the application is restarting. Then I check whether the issue is caused by the application, configuration, health probes, or resource limits.
I also compare the new Deployment configuration with the previous working version. After identifying the root cause, I correct the configuration or application issue and verify that the restart count is stable.
Simple ga gurthupettuko: Deployment tarvatha app restart avutunte → first Pod restart count and logs chudu → application error unda check cheyyi → configuration and probes verify cheyyi → resource limits check cheyyi → previous working version tho compare cheyyi → issue fix chesi restart count stable ga unda verify cheyyi.
Eng: If the application restarts after deployment → first check Pod restart count and logs → check application errors → verify configuration and probes → check resource limits → compare with the previous working version → fix the issue and verify stable restarts.
Example:
kubectl get pods → Pod restart count increase avutunda chudu.
kubectl logs <pod-name> --previous → restart mundu application error em vachindo check cheyyi.
kubectl describe pod <pod-name> → Events lo probe or resource issue unda chudu.
kubectl get deployment <deployment-name> -o yaml → latest configuration verify cheyyi.
Issue fix chesi → kubectl get pods → restart count stable ga unda verify cheyyi.

---

18. Kubernetes Pod is running but shows 0/1 Ready. What will you check?
Ans: First, I check the readiness probe configuration and Pod events to understand why the container is not becoming Ready. Then I verify the application port, health endpoint, and response expected by the probe.
I also check whether the application startup is taking longer than the configured initial delay. After correcting the probe or application issue, I verify that the Pod changes to Ready and starts receiving traffic.
Simple ga gurthupettuko: Pod 0/1 Ready unte → first readiness probe and events chudu → application port and health endpoint verify cheyyi → expected response vastunda check cheyyi → startup time ekkuva unda chudu → initial delay verify cheyyi → issue fix chesi Pod Ready ayinda check cheyyi.
Eng: If the Pod shows 0/1 Ready → first check readiness probe and events → verify application port and health endpoint → check the expected response → verify startup time and initial delay → fix the issue and confirm the Pod becomes Ready.
Example:
kubectl get pods → Pod 0/1 Ready lo unda chudu.
kubectl describe pod <pod-name> → readiness probe failure reason Events lo chudu.
kubectl get pod <pod-name> -o yaml → readiness path, port and initialDelaySeconds verify cheyyi.
kubectl exec -it <pod-name> -- curl localhost:<port>/<health-path> → health endpoint response check cheyyi.
Issue fix chesi → kubectl get pods → 1/1 Ready ayyinda verify cheyyi.

---

19. Kubernetes Deployment is creating old and new Pods together. What will you check?
Ans: First, I check the Deployment rollout strategy and determine whether it is using RollingUpdate. Then I verify the maxSurge and maxUnavailable settings because they control how many old and new Pods can exist during the rollout.
I also check whether the new Pods are becoming Ready successfully. If the rollout is stuck, I review Deployment events and Pod status to identify the reason.
Simple ga gurthupettuko: Old and new Pods rendu kalisi unte → first Deployment strategy chudu → RollingUpdate aa check cheyyi → maxSurge and maxUnavailable verify cheyyi → new Pods Ready avutunnaya chudu → rollout stuck ayithe events and Pod status check cheyyi.
Eng: If old and new Pods are running together → first check the Deployment strategy → verify RollingUpdate → check maxSurge and maxUnavailable → confirm new Pods become Ready → if rollout is stuck, check events and Pod status.
Example:
kubectl get deployment <deployment-name> -o yaml → Deployment strategy RollingUpdate aa chudu.
kubectl describe deployment <deployment-name> → rollout details and events check cheyyi.
kubectl get pods → old and new Pods status chudu.
kubectl rollout status deployment/<deployment-name> → rollout complete ayyinda check cheyyi.
maxSurge and maxUnavailable values required prakaram correct chesi → rollout malli verify cheyyi.

---

20. Kubernetes Pod is unable to communicate with another Pod. What will you check?
Ans: First, I check whether both Pods are running and whether they are using the expected IP addresses or Service names. Then I verify the Service configuration if communication is through a Service.
I also check NetworkPolicies because they can restrict communication between Pods or namespaces. Finally, I test connectivity from inside the Pod and review application and network-related events.
Simple ga gurthupettuko: One Pod nunchi another Pod ki communication avvakapothe → first rendu Pods running unnaya chudu → Pod IP or Service name correct aa check cheyyi → Service configuration verify cheyyi → NetworkPolicy block chesthunda chudu → Pod lopala connectivity test cheyyi → network issue identify chesi fix cheyyi.
Eng: If one Pod cannot communicate with another → first check both Pods → verify Pod IP or Service name → check Service configuration → verify NetworkPolicy restrictions → test connectivity from inside the Pod → identify and fix the network issue.
Example:
kubectl get pods -o wide → rendu Pods running unnaya and IP addresses chudu.
kubectl get svc <service-name> → Service name and targetPort correct ga unnaya check cheyyi.
kubectl exec -it <source-pod> -- curl <service-name>:<port> → source Pod nunchi destination Service ki connectivity test cheyyi.
kubectl get networkpolicy → Pod-to-Pod traffic block chesthunda check cheyyi.
Issue fix chesi → same curl command run chesi → destination application response vastunda verify cheyyi.